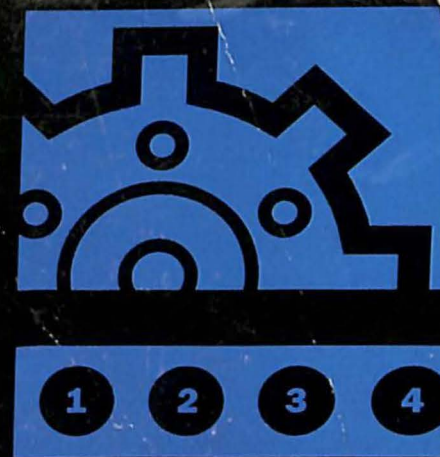
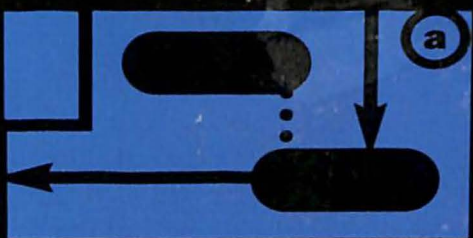
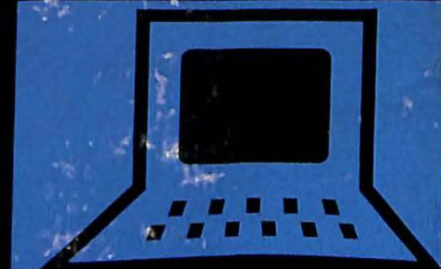
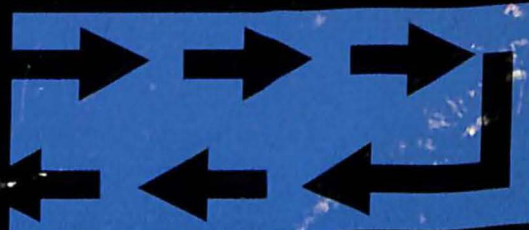
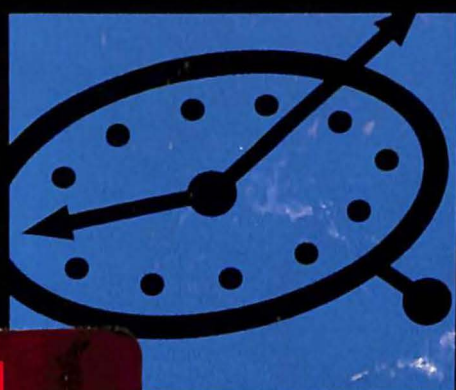


$$\Gamma_L(f) = \frac{Z_L(f)}{Z(f)}$$



# Programación en Linux

C O N E J E M P L O S



```
printf("\n\n\n\n\nWhat is the  
name? "); gets(item.name);  
printf("What is the city?  
"); gets(item.city);  
getzip(item.zipcode); ans = g  
char(); if((fp=fopen(FILENAME,  
"r+")) == NULL){ err_msg("F  
Read error-ensure file exists
```



Prentice  
Hall

que®

Kurt Wall

**KURT WALL**

# **Programación en Linux**

**C O N   E J E M P L O S**



**BIBLIOTECA  
UNIVERSIDAD de PALERMO**

*Prohibida su Reproducción - Ley 11723*



**Argentina • Bolivia • Brasil • Colombia • Costa Rica • Chile • Ecuador • Salvador •  
España • Guatemala • Honduras • México • Nicaragua • Panamá • Paraguay • Perú •  
Puerto Rico • República Dominicana • Uruguay • Venezuela**

**Amsterdam • Harlow • Londres • Menlo Park • Miami • Munich • Nueva Delhi • Nueva Jersey •  
Nueva York • Ontario • París • Singapur • Sydney • Tokio • Toronto • Zurich**

datos de catalogación bibliográfica

519.68  
WAL

Wall, Kurt

Programación en Linux con ejemplos - 1ª ed -

Buenos Aires: Prentice Hall, 2000

568 p.; 24x19 cm

Traducción de: Jorge Gorín

ISBN: 987-9460-09-X

I. Título - 1. Programación

UNIVERSIDAD DE PALERMO  
BIBLIOTECA

042613 ✓

Fecha recepción: 08.10.03

Procedencia: COMPRA

Editora: María Fernanda Castillo  
Armado de interior y tapa: Pérez Villamil & Asociados  
Traducción: Jorge Gorín  
Producción: Marcela Mangarelli

Traducido de:  
Linux Programming by example, by Kurt Wall, published by QUE.  
Copyright © 2000,  
All Rights Reserved.  
Published by arrangement with the original publisher,  
PRENTICE HALL, inc, A Pearson Education Company.  
ISBN: 0-7897-2215-1

Edición en Español publicada por Pearson Education, S.A.  
Copyright © 2000  
ISBN: 987-9460-09-X

Este libro no puede ser reproducido total ni parcialmente en ninguna forma, ni por ningún medio o procedimiento, sea reprográfico, fotocopia, microfilmación, mimeográfico o cualquier otro sistema mecánico, fotoquímico, electrónico, informático, magnético, electroóptico, etcétera. Cualquier reproducción sin el permiso previo por escrito de la editorial viola derechos reservados, es ilegal y constituye un delito.

© 2000, PEARSON EDUCATION S.A.  
Av. Regimiento de Patricios 1959 (1266), Buenos Aires,  
República Argentina

Queda hecho el depósito que dispone la ley 11.723  
Impreso en Perú. Printed in Perú.  
Impreso en Quebecor Perú, en el mes de noviembre de 2000  
Primera edición: Diciembre de 2000

**Associate Publisher**  
Dean Miller

**Executive Editor**  
Jeff Koch

**Acquisitions Editor**  
Gretchen Ganser

**Development Editor**  
Sean Dixon

**Managing Editor**  
Lisa Wilson

**Project Editor**  
Tonya Simpsons

**Copy Editor**  
Kezia Endsley

**Indexer**  
Cheryl Landes

**Proofreader**  
Benjamin Berg

**Technical Editor**  
Cameron Laird

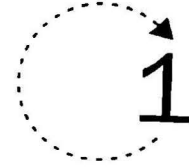
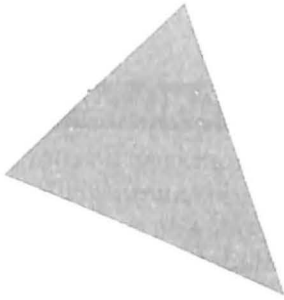
**Team Coordinator**  
Cindy Teeters

**Interior Design**  
Karen Ruggles

**Cover Design**  
Duane Rader

**Copy Writer**  
Eric Borgert

**Production**  
Dan Harris  
Heather Moseman



## Compilación de programas

El GNU cc (gcc) es el conjunto de programas que comprenden el compilador del proyecto GNU. Compila programas escritos en C, C++, o C dirigido a Objetos. El gcc también compila FORTRAN (con apoyo de g77). Este capítulo se concentra en el compilador de C porque C es la lengua nativa de Linux. El capítulo también cubre las diferencias entre gcc y egcs versión 1.1.2, el conjunto experimental de programas del compilador GNU.

Este capítulo abarca los siguientes temas:

- Invocación del gcc
- Opciones y argumentos para controlar la conducta del gcc
- Compilación de múltiples archivos fuente
- Utilización de las prestaciones de optimización de gcc
- Utilización de las prestaciones de depuración de gcc
- Administración de errores de compilación
- Extensiones del GNU al lenguaje C
- El nuevo compilador de C, egcs

Todos los programas de este capítulo pueden ser encontrados en el sitio web <http://www.mcp.com/info> bajo el número de ISBN 0789722151.

## Utilización del compilador de código de GNU (gcc)

El gcc le da al programador un amplio control sobre el proceso de compilación. Este último comprende cuatro etapas: *preprocesado*, *compilación*, *ensamblaje* y *vinculación*. Uno puede detener el proceso después de cualquiera de dichas etapas para examinar el rendimiento del compilador en la misma. El gcc también puede administrar los diversos dialectos de C, tales como el ANSI C o el C tradicional (el de sus desarrolladores, Kernighan y Ritchie). Se puede controlar la cantidad y el tipo de información de depuración, si se la deseara emplear, que se incrustará en el archivo binario resultante y, como la mayoría de los compiladores, gcc puede llevar a cabo también optimización de código.

El gcc incluye más de treinta advertencias individuales y tres niveles de advertencia tipo "captura todo". El gcc es también un compilador cruzado, de modo que se pueda desarrollar el código en una arquitectura de procesador y correrlo en otra. La compilación cruzada es importante porque Linux corre en muchos tipos diferentes de sistema, tales como el x86s de Intel, las PowerPC, las Amiga y las SCN Sparcs. Cada chip de procesador tiene una arquitectura física diferente, de modo que la manera en que se debe construir un binario varía para cada sistema. Cuando se utiliza como un compilador cruzado, el gcc permite que se compile un programa diseñado para correr en una PowerPC en, digamos, un sistema Intel x86.

Finalmente, el gcc acepta una lista larga de extensiones a C. La mayoría de estas extensiones mejora el desempeño, contribuye a la tarea del compilador tendiente a la optimización del código, o hace más sencillo el trabajo del programador. El precio a pagar a cambio es la disminución de la portabilidad. Mencionaremos algunas de las extensiones más comunes, porque las mismas se pueden encontrar en los archivos de encabezamiento del kernel, pero lo ideal es que el lector evite su uso en los programas que redacte.

### Invocación de gcc

Para utilizar gcc, suministre un nombre de archivo fuente de C y utilice la opción `-o` para especificar el nombre del archivo de salida. gcc preprocesará, compilará, ensamblará y vinculará (*link*) el programa, generando un archivo ejecutable, a menudo denominado binario. La sintaxis más simple se ilustra aquí:

```
gcc archivo_entrada.c [-o archivo_salida]
```

`archivo_entrada.c` es un archivo de código fuente en C y `-o` establece que el nombre del archivo de salida será `archivo_salida`. Los corchetes (`[ ]`) indican a lo largo de este libro argumentos opcionales. Si el nombre del archivo de salida no se especifica, gcc lo denominará `a.out` como opción predeterminada.



EJEMPLO

### Ejemplo

Este ejemplo utiliza gcc para crear el programa `hola` a partir del código fuente `hola.c`. Primero, el código fuente:

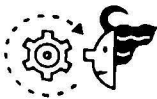
```
/* Nombre del programa en Internet: hello.c */
/*
 * Listado 1.1
```

```
* hola.c - Programa canónico "¡Hola, mundo!"
*/
#include <stdio.h>

int main(void)
{
    puts("¡Hola, mundo de programadores de Linux!");
    return 0;
}
```

para compilar y correr este programa, tipee  
\$ gcc hola.c -o hola

Si todo anduvo bien, gcc realiza su trabajo en silencio y luego regresa a la señal de petición de comandos del sistema operativo. gcc compila y linkea el archivo fuente `hola.c`, creando el archivo binario que fue especificado mediante el argumento `-o`, o sea `hola`.



**SALIDA**

```
El comando ls (que vendría a ser algo así como la contraparte UNIX del co-
mando DIR de DOS) muestra que existe un nuevo programa, hola. El últi-
mo comando ejecuta el programa, y produce la siguiente salida:

$ ls
hola.c      hola*
$ ./hola
¡Hola, mundo de programadores de Linux!
```

**PRECAUCIÓN**

El comando que ejecutó el programa `hola` incluía específicamente el directorio corriente, representado con un punto (`.`), porque incluir el directorio corriente en la ruta representa un riesgo para la seguridad. Es decir, si el directorio corriente fuese un subdirectorio del último directorio incluido en la ruta, en lugar de utilizar una sentencia `$PATH` semejante a `/bin:/usr/bin:/usr/local/bin:.` (con el punto al final), la sentencia debería ser `/bin:/usr/bin:/usr/local/bin`, de modo que ningún hacker pudiera colocar en el directorio corriente, un comando dañino del mismo nombre del comando normal que uno verdaderamente desea ejecutar.

¿Cómo sabe gcc cómo procesar los archivos? Se basa en las extensiones de los mismos para determinar cómo procesar correctamente cada archivo. Las extensiones más comunes y sus interpretaciones se listan en la tabla 1.1.

**Tabla 1.1.** *Cómo interpreta gcc las extensiones de archivo*

| Extensión | Tipo de archivo                 |
|-----------|---------------------------------|
| .c        | Código fuente de lenguaje C     |
| .C, .cc   | Código fuente de lenguaje C++   |
| .i        | Código fuente de C preprocesado |

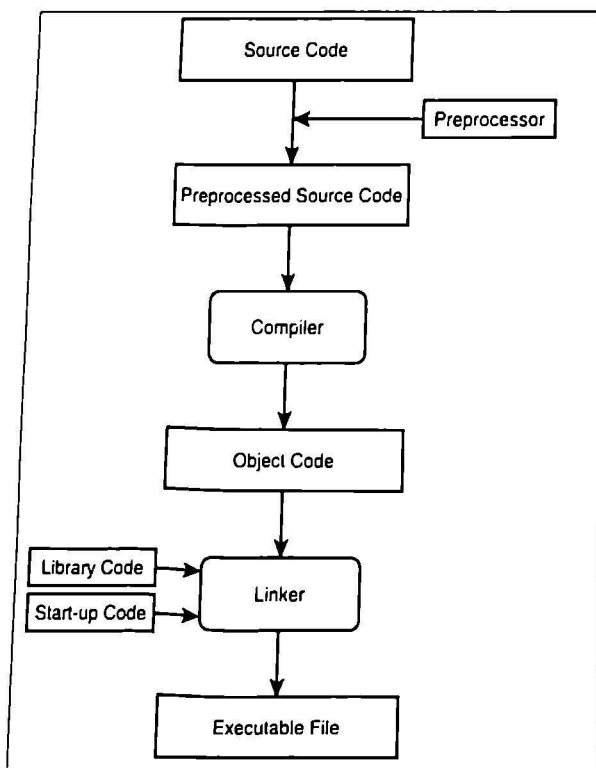
*continúa*

**Tabla 1.1.** continuación

| Extensión | Tipo de archivo                       |
|-----------|---------------------------------------|
| .ii       | Código fuente de C++ preprocesado     |
| .S, .s    | Código fuente de lenguaje ensamblador |
| .o        | Código objeto compilado               |
| .a, .so   | Código de biblioteca compilado        |

### Invocación de gcc paso a paso

En el primer ejemplo, hubo muchas cosas que tuvieron lugar de manera encubierta y el lector no las vio. gcc primero corrió `hola.c` por el preprocesador, `cpp`, para expandir cualquier macro que pudiese haber e insertar los contenidos de los archivos incluidos mediante `#include`. Luego compiló el código fuente preprocesado, convirtiéndolo en código objeto. Finalmente el linker, `ld`, creó el archivo binario `hola`. El proceso completo de vinculación se ilustra en la figura 1-1.



**Figura 1.1.** La compilación de un programa consiste de varias etapas.

Uno puede recrear estas etapas manualmente, avanzando por el proceso de compilación un paso por vez. Para indicarle a gcc que detenga la compilación luego del preprocesado, utilice la opción `-E` de gcc, como se indica a continuación:

```
$ gcc -E archivo_entrada.c -o archivo_salida.cpp
```

El paso siguiente consiste en compilar el archivo preprocesado para convertirlo en código objeto, utilizando la opción `-c` de `gcc`. La opción `-x` se utiliza para indicarle a `gcc` que comience la compilación a partir de cierta etapa. La sintaxis correcta para esta etapa es:

```
$ gcc -x cpp-output -c archivo_entrada.cpp -o archivo_salida.o
```

*Linkeando* el archivo objeto, finalmente, se obtiene su imagen binaria. El comando que llevaría a cabo la etapa de vinculación sería algo así como la siguiente:

```
$ gcc archivo_entrada.o -o archivo_salida
```

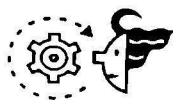
**EJEMPLO****Ejemplo**

Volviendo al programa `hola` del ejemplo anterior, avance paso a paso a través del proceso de compilación tal como lo ilustra el ejemplo siguiente.

Primero, preprocese `hola.c`:

```
$ gcc -E hola.c -o hola.cpp
```

Si el lector examina `hola.cpp` con un editor de texto, verá que el contenido del archivo de encabezamiento `stdio.h` ha sido efectivamente insertado en el código fuente, junto a otros símbolos de preprocesado. El listado siguiente es un extracto de `hola.cpp`:

**SALIDA**

```
extern int fgetpos (FILE * stream, fpos_t * _pos)    ;

extern int fsetpos (FILE *__stream, const fpos_t *__pos)    ;

# 519 "/usr/include/stdio.h" 3
# 529 "/usr/include/stdio.h" 3

extern void clear_err (FILE *__stream)    ;

extern int feof (FILE *__stream)    ;

extern int ferror (FILE *__stream)    ;

extern void clearerr_unlocked (FILE *__stream) ;
extern int feof_unlocked (FILE *__stream)    ;
extern int ferror_unlocked (FILE *__stream)    ;

extern void perror (__const char *__s)    ;

extern int sys_nerr;
extern __const char *__const sys_errlist[];
```

El paso siguiente es compilar `hola.cpp` para convertirlo en código objeto:

```
$ gcc -x cpp-output -c hola.cpp -o hola.o
```

Se debe utilizar la opción `-x` para indicarle a gcc que comience la compilación a cierta altura, en este caso, a partir del código fuente preprocesado. Cuando se efectúa el linkeo del código objeto, finalmente, se crea un archivo binario:

```
$ gcc hola.o -o hola
```

Espero que con esto el lector pueda advertir que es mucho más sencillo utilizar la sintaxis “abreviada”, `gcc hola.c -o hola`. El ejemplo paso a paso demostró que uno puede detener y comenzar la compilación en cualquier etapa, si surgiese la necesidad.

Una situación en la cual uno puede realizar la compilación paso a paso es cuando se está creando bibliotecas. En este caso, uno solo desea crear archivos objeto, de modo que la etapa final de linkeo resulta innecesaria. Otra circunstancia de realizar el proceso de compilación paso a paso es cuando un archivo incluido mediante `#include` produce conflictos con el código que el lector ha redactado, o tal vez con otro archivo insertado con `#include`. Recorrer el proceso paso a paso permitirá ver con mayor claridad cuál es el archivo que genera el conflicto.

### Utilización de múltiples archivos de código fuente

La mayoría de los programas de C consiste de múltiples archivos de código fuente, de modo que cada archivo fuente debe ser compilado a código objeto antes del paso final de linkeo. Este requerimiento se satisface con facilidad. Sólo se requiere proveer a gcc del nombre de cada archivo de código fuente que se deba compilar. gcc se hace cargo del resto del proceso. La invocación de gcc sería similar a la siguiente:

```
$ gcc archivo1.c archivo2.c archivo3.c -o nombre_programa
```

gcc primero crea `archivo1.o`, `archivo2.o` y `archivo3.o` antes de linkearlos entre sí para crear `nombre_programa`.



**EJEMPLO**

### Ejemplo

Supongamos que se desee compilar `nuevo-hola`, que utiliza código proveniente de dos módulos, `mostrar.c` y `mensajes.c`, y un archivo de encabezado, `msg.h`:

```
/* Nombre de programa en Internet: showit.c
/* nuevo_hola.c */
/* -----mostrar.c----- */
/*
 * mostrar.c _ Controlador de pantalla
 */
#include <stdio.h>
#include "mensaje.h"
int main(void)
{
    char mensaje_bienvenida[] = {"¡Hola, programador!"};
    char mensaje_despedida[] = {"¡Adiós, programador!"};
```

```

        printf("%s\n", mensaje_bienvenida);
        imprimir_mensaje(mensaje_despedida);
        return 0;
    }
    /* ----- mensaje.h
    ----- */

    /*
    * mensaje.h - Encabezado de mensaje.c
    */

    #ifndef MENSAJE_H_
    #define MENSAJE_H_

    void imprimir_mensaje(char *mensaje);

    #endif /* MENSAJE_H_ */
    /* ----- mensaje.c
    ----- */

    /*
    * mensaje.c - Funcion definida en mensaje.h
    */
    #include <stdio.h>
    #include "mensaje.h"

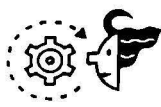
    void imprimir_mensaje(char *mensaje)
    {
        printf("%s\n", mensaje);
    }

```

El comando para compilar estos programas, y por lo tanto para crear nuevo\_hola, es:

```
$ gcc mensaje.c mostrar.c -o nuevo_hola
```

Al correr este programa, su salida será la siguiente:



**SALIDA**

```
$ ./nuevo_hola
```

```
¡Hola, programador!
```

```
¡Adios, programador!
```

gcc recorre las mismas etapas de preprocesado-compilación-linkado que antes, esta vez creando archivos objeto para cada archivo fuente antes de pro-

ceder a crear el archivo final binario, nuevo\_hola. Tipear comandos largos como éste puede volverse tedioso, sin embargo. En el capítulo 2, “Control del proceso de compilación: el make de GNU”, se verá cómo resolver este problema. La próxima sección presentará las muchas opciones de línea de comando de gcc.

Opciones y argumentos

La lista de opciones de línea de comando que acepta gcc ocuparía varias páginas, de modo que la tabla 1.2 presenta sólo las más comunes.

Tabla 1.2. Opciones de línea de comando de gcc

| Opción           | Propósito                                                                                                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -o nom_archivo   | Designa el archivo de salida nom_archivo (no necesariamente cuando se compila código objeto). Si no se especifica el nombre de este archivo, la opción predeterminada es a.out.                                            |
| -c               | Compilar sin <i>linkear</i> .                                                                                                                                                                                              |
| -Dfoo=bar        | Define un macro de preprocesador foo con un valor bar en la línea de comandos.                                                                                                                                             |
| -Iruta_dir       | Sitúa la ruta de acceso al directorio especificado en ruta_dir al comienzo de la lista de directorios en que gcc busca los archivos a incluir por la presencia de directivas <i>#include</i> .                             |
| -Lruta:dir       | Sitúa la ruta de acceso al directorio especificado en ruta_dir al comienzo de la lista de directorios en que gcc busca archivos de biblioteca. Como opción predeterminada, gcc linkea bibliotecas compartidas.             |
| -static          | Linkea bibliotecas estáticas.                                                                                                                                                                                              |
| -lfoo            | Linkea la biblioteca foo.                                                                                                                                                                                                  |
| -g               | Incluye en el archivo binario información estándar de depuración.                                                                                                                                                          |
| -ggdb            | Incluye en el archivo binario muchísima información de depuración que sólo puede interpretar el depurador del GNU, gdb.                                                                                                    |
| -O               | Optimiza el código compilado. Esto es equivalente a especificar -O1.                                                                                                                                                       |
| -On              | Especifica un nivel de optimización n, 0<=n<=3.                                                                                                                                                                            |
| -ansi            | Coloca el compilador en modo ANSI/ISO C, no permitiendo extensiones GNU que generen conflictos con dichos estándares.                                                                                                      |
| -pedantic        | Muestra todas las advertencias que requiere el ANSI/ISO C estándar.                                                                                                                                                        |
| -pedantic-errors | Muestra todos los errores que requiere el ANSI/ISO C estándar.                                                                                                                                                             |
| -traditional     | Habilita el apoyo para la sintaxis del C de Kernighan y Ritchie (si el lector no comprende lo que significa esto, no se preocupe por ello; basta con saber que Kernighan y Ritchie fueron los diseñadores del C original). |
| -W               | Suprime todos los mensajes de advertencia.                                                                                                                                                                                 |
| -Wall            | Exhibe todas las advertencias generalmente aplicables que puede proveer gcc. Para ver advertencias específicas, utilice -Wwarning.                                                                                         |

Continúa

**Tabla 1.2.** Continuación

| Opción  | Propósito                                                                                                     |
|---------|---------------------------------------------------------------------------------------------------------------|
| -werror | En lugar de generar advertencias, gcc convertirá las advertencias en errores, deteniendo la compilación.      |
| -MM     | Emitirá una lista de dependencia compatible con make. útil para crear una makefile en un proyecto complicado. |
| -v      | Muestra los comandos utilizados en cada etapa de la compilación.                                              |

El lector ya ha visto cómo operan `-c` y `-o`. Si no se especifica `-o`, sin embargo, para un archivo de entrada denominado `nom_archivo.ext` (donde `ext` representa cualquier extensión de hasta tres caracteres), la acción predeterminada de gcc será denominar al archivo ejecutable `a.out`, al archivo objeto `nom_archivo.o`, y al archivo en lenguaje ensamblador `nom_archivo.s`. La salida del preprocesador va al dispositivo que se encuentre establecido como `stdout` (dispositivo estándar de salida).

### ARCHIVOS DE BIBLIOTECA Y DE INCLUSIÓN

Si se dispone de archivos de biblioteca o de inclusión situados en ubicaciones no estándar, las opciones `-Lruta_dir` y `-Iruta_dir` permiten especificar dichas ubicaciones y asegurar que se busquen los respectivos archivos en esas ubicaciones antes de hacerlo en las estándar (las asignadas por el compilador como predeterminadas). Por ejemplo, si se almacenan los archivos de inclusión habituales en `/usuario/local/mis_inclusiones`, para que gcc los encuentre la invocación a gcc debería ser algo parecido a:

```
$ gcc infile.c -I/usuario/local/mis_inclusiones
```

De manera similar, supongamos que el lector esté comprobando una nueva biblioteca de programación, `libnueva.a`, que está corrientemente almacenada en `/usuario/local/mis_bibliotecas`, antes de instalarla como biblioteca estándar de sistema. Supongamos también que los archivos de encabezado están almacenados en `/usuario/local/mis_inclusiones`. Consecuentemente, para linkear `libnueva.a` y ayudar a gcc a encontrar los archivos de encabezado, el comando gcc de línea de comandos debería ser similar al siguiente:

```
$ gcc mi_aplicacion.c -I/usuario/local/mis_inclusiones -L/usuario/local/mis_bibliotecas -lnueva
```

La opción `-l` le indica al linker que tome código objeto desde la biblioteca especificada. El ejemplo anterior vincularía `libnueva.a`.

### CONSEJO

Una convención UNIX de vieja data requiere que las bibliotecas se denominen `lib{algún nombre}`, y gcc, al igual que la mayoría de los compiladores de UNIX y de Linux, se basa en dicha convención. Si el lector no utiliza la opción `-l` cuando vincula bibliotecas o no provee una ruta de acceso a la biblioteca a ser vinculada, la etapa de linkeo no tendrá éxito y gcc emitirá un mensaje de error aduciendo referencias sin definir a "nombre\_de\_función".

Como opción predeterminada gcc utiliza bibliotecas compartidas, de modo que si el lector necesitara vincular bibliotecas estáticas, deberá utilizar la opción `-static`. Esto significa que sólo se utilizarán bibliotecas estáticas. El siguiente ejemplo crea un archivo ejecutable vinculado contra la biblioteca estática `ncurses`:

```
$ gcc aplic_curses.c -lncurses -static
```

✓ Para aprender a programar con `ncurses`, ver "Una interfaz API de usuario en modo texto", pág. 219.

Cuando uno vincula bibliotecas estáticas, el archivo ejecutable que se obtiene resulta mucho más grande que cuando se utilizan bibliotecas compartidas. ¿Para qué utilizar una biblioteca estática, entonces? Una razón habitual es para garantizar que los usuarios puedan utilizar el programa: en el caso de las bibliotecas compartidas, el código que necesita un programa para poder correr se linkea al mismo dinámicamente durante su ejecución, en lugar de serlo estáticamente durante su compilación. Si la biblioteca compartida que requiere el programa para poder ser corrido no se encuentra instalada en el sistema del usuario, el mismo obtendrá errores y no podrá correr el programa.

El navegador Netscape constituye un ejemplo perfecto de este problema. Netscape se basa fuertemente en Motif, un costoso toolkit de programación en X. La mayoría de los usuarios de Linux no pueden afrontar el costo de instalar Motif en sus sistemas porque el mismo cuesta demasiado dinero, de modo que Netscape en la práctica instala dos versiones de su navegador en su sistema; una que linkea bibliotecas compartidas, `netscape-dynMotif`, y una que vincula bibliotecas estáticas, `netscape-statMotif`. El propio archivo ejecutable de Netscape es en realidad un *script* de interfaz que determina si el usuario posee instalada la biblioteca compartida de Motif y activa uno u otro de los archivos binarios, según sea requerido.



EJEMPLO

### Ejemplos

1. Este ejemplo crea una biblioteca estática, `libmensajes.a`. Los comandos requeridos para hacer esto son:

```
$ gcc -c mensajes.c -o libmensajes.o
$ ar rcs libmensajes.a libmensajes.o
```

Recuérdese en un párrafo anterior de este capítulo que `-c` le indica a `gcc` crear un archivo objeto denominado, en este caso, `libmensajes.o`. la segunda línea utiliza el comando `ar` para crear una biblioteca estática denominada `libmensajes.a` a partir del módulo objeto, `libmsg.o`. El ejemplo siguiente utiliza esa biblioteca.

✓ Para aprender más sobre el comando `ar`, ver “El comando `ar`”, pág. 301.

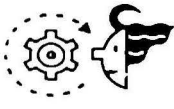


EJEMPLO

2. Ahora el lector dispone de una biblioteca para poder ser linkeada. Le deberá informar a `gcc` dónde encontrar el archivo de inclusión, utilizando `-I`, dónde encontrar la biblioteca, empleando `-L`, y el nombre de la biblioteca, utilizando `-l`, como se ilustra aquí:

```
$ gcc mostrar.c -o nuevo_hola_lib -I. -L. -lmensajes
```

Un ejemplo anterior creaba `nuevo_hola` compilando juntos `mostrar.c` y `mensajes.c`. Esta vez, creamos `nuevo_hola_lib` linkeando la biblioteca estática `libmensajes.a`. Este ejemplo creó un archivo ejecutable denominado `nuevo_hola-lib` (el nombre fue elegido aquí para distinguirlo del que se empleaba en el otro ejemplo) vinculando a `libmensajes.a`, (`-lmensajes`) ubicado en el directorio de trabajo corriente (`-L.`). El punto (`.`) situado al lado de la letra `L` representa el directorio corriente. La salida de este programa se lista a continuación:



**SALIDA**

```
$ ./nuevo_hola_lib
¡Hola, programador!
¡Adios, programador!
```

## VERIFICACIÓN DE ERRORES Y AVISOS

gcc ofrece toda una gama de opciones de línea de comando para verificación de errores y generación de avisos. Éstas incluyen `-ansi`, `-pedantic`, `-pedantic-errors`, y `-Wall`. A modo de comienzo, `-pedantic` le indica a gcc emitir todas las advertencias requeridas por el estricto ANSI/ISO C estándar. Todo programa que utilice extensiones prohibidas, tales como los aceptados por gcc, será rechazado. `-pedantic-errors` se comporta de manera similar, excepto que emite mensajes de error en lugar de advertencias. `-ansi` deshabilita las extensiones GNU que no se ajusten al estándar. Ninguna de esas opciones, sin embargo, garantiza que un código fuente, aun cuando compile sin errores al utilizar algunas o todas estas opciones, sea ciento por ciento concordante con ANSI/ISO.

La opción `-Wall` instruye a gcc a que exhiba todas las advertencias generalmente relevantes que pueda en relación al código que está siendo compilado. `-Wall` emite avisos sobre código que la mayoría de los programadores consideran objetables y/o que sean sencillos de modificar para evitar la generación de la advertencia. Un ejemplo de mala práctica de programación es declarar una variable pero no emplearla. Otro es declarar una variable sin establecer explícitamente su tipo.

## Ejemplos

Consideremos el siguiente ejemplo, que muestra una muy mala práctica de programación. En el mismo se declara `main` como retornando `void`, cuando de hecho `main` retorna `int`, y utiliza la extensión GNU `long long int` para declarar un entero de 64 bits.

```
/* Nombre del programa en Internet: pedant.c */
/*
 * pedant.c - Compilar sucesivamente utilizando primero
 * -ansi, luego -pedantic y finalmente -pedantic-errors
 */
#include <stdio.h>

void main(void)
{
    long long int i = 01;
    puts ("Este no es un programa que cumpla con las reglas de C");
}
```

**EJEMPLO**

1. Primero, trate de compilarlo utilizando verificación de no conformidad:

```
$ gcc pedant.c -o pedant
```

```
pedant.c: In function 'main':
```

```
pedant.c:8: warning: return type of 'main' is not 'int'
```

**SALIDA**

2. Luego, compile utilizando el especificador `-ansi`:

```
$ gcc -ansi pedant.c -o pedant
```

```
pedant.c: In function 'main':
```

```
pedant.c:8: warning: return type of 'main' is not 'int'
```

**SALIDA**

Lo que muestra esta salida es que `-ansi` obliga a gcc a emitir los mensajes de diagnóstico requeridos por el estándar. No garantiza que su código responda a las normas de ANSI C. El programa compilará a pesar de la incorrecta declaración de `main`.

3. Ahora, utilice `-pedantic`:

```
$ gcc -pedantic pedant.c -o pedant
```

```
pedant.c: In function 'main':
```

```
pedant.c:9: warning: ANSI C does not support 'long'
```

```
pedant.c:8: warning: return type of 'main' is not 'int'
```

**SALIDA**

Nuevamente el programa compilará, a pesar de las advertencias emitidas.

4. Finalmente, utilice `-pedantic-errors`:

```
$ gcc -pedantic-errors pedant.c -o pedant
```

```
pedant.c: In function 'main':
```

```
pedant.c:9: ANSI C does not support 'long long'
```

```
pedant.c:8: return type of 'main' is not 'int'
```

**SALIDA**

Esta vez el programa no se compila. gcc se detiene después de exhibir los diagnósticos de error requeridos.

A riesgo de parecer reiterativo, las opciones de compilador `-ansi`, `-pedantic` y `-pedantic-errors` no aseguran que se produzca un código que cumpla con las normas ANSI/ISO. Las mismas meramente lo guían durante la compilación. Resulta instructivo señalar el comentario que aparece en la documentación de gcc sobre el uso de `-pedantic`:

“Esta opción no se pretende que sea útil; existe solo para satisfacer a pedantes que de no ser así alegarían que GNU CC no responde al estándar ANSI. Algunos usuarios tratan de utilizar `-pedantic` para verificar la estricta adherencia de sus programas al ANSI C. Pronto encuentran que no hace del todo lo que desean: detecta algunas prácticas no ANSI, pero no todas; solo aquellas para las cuales ANSI C requiere un diagnóstico.”

**OPCIONES DE OPTIMIZACIÓN**

La optimización del código es una tentativa de mejorar el desempeño de un programa. La desventaja consiste en mayores tiempos de compilación, mayor empleo de la memoria durante la compilación y, tal vez, mayor tamaño de código.

Actualmente, gcc tiene tres niveles de optimización. La opción `-O` sin aditamentos le indica a gcc que reduzca tanto el tamaño del código como el tiempo de la ejecución. Es equivalente a `-O1`. Los tipos de optimización realizados a este nivel dependen del procesador al que se apunta, pero siempre incluyen por lo menos optimización de hilos de comandos `jump` (*saltar a*) y diferimiento de comandos `pop` (*retorno de valores de registros*) desde la pila (*stack*). Las optimizaciones de hilos de comandos `jump` intentan reducir el número de este tipo de operaciones. Los diferimientos de pops desde la pila tienen lugar cuando el compilador permite que los argumentos se acumulen en la pila a medida que las funciones retornan valores y luego los regresa simultáneamente, en lugar de ir retornando los argumentos de a uno por vez, cuando retorna cada función llamada.

Las optimizaciones de nivel `-O2` incluyen todas las optimizaciones de nivel `O1` más toques adicionales que se vinculan con la manera en que se llevan a cabo los comandos del procesador. En este nivel de optimización, el compilador intenta garantizar que el procesador cuente con comandos para ejecutar mientras espera por los resultados de otros comandos. El compilador también intenta compensar por la tardanza de datos, que es la demora ocasionada por el tiempo que requiere obtener datos desde el “escondite” o la memoria principal. La implementación de esta optimización depende en gran parte del tipo de procesador sobre el que se realice.

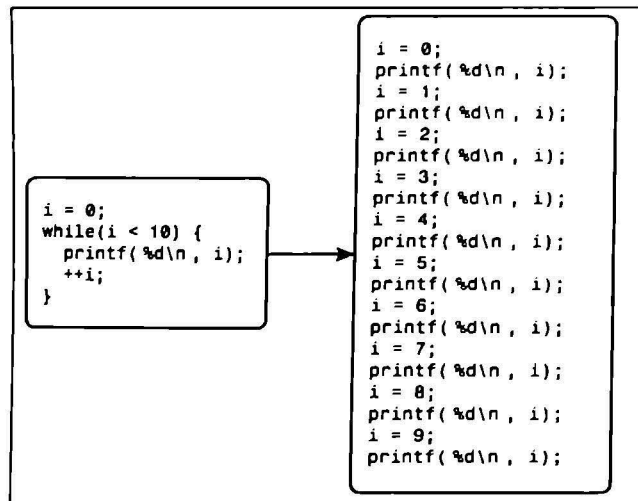
Las opciones `-O3` incluyen todas las optimizaciones `-O2`, desenrollo de lazos y otras prestaciones muy vinculadas con el tipo de procesador.

Según sea la cantidad de conocimiento a bajo nivel que uno tenga sobre una familia dada de CPU, podrá utilizar la opción `-fflag` para requerir las optimizaciones específicas que se desea llevar a cabo. Tres de esos indicadores (*flags*) merecen consideración: `-ffastmath`, `-finline-functions` y `-funroll-loops`. `-ffastmath` genera optimizaciones sobre las operaciones matemáticas de punto flotante que aumentan la velocidad, pero violan estándares IEEE y/o ANSI. `-finline-functions` expande todas las funciones “simples” durante la compilación de un programa, de manera similar a los reemplazos de los macros en el preprocesador. Es el compilador, sin embargo, el que decide cuál constituye una función simple y cuál no.

`-funroll-loops` instruye a gcc para que desenrolle todos los lazos que tengan un número fijo de iteraciones, siempre y cuando dicho número de iteraciones pueda ser determinado durante la compilación. Desenrollar un lazo significa que cada iteración del mismo se convertirá en una sentencia individual. Así, por ejemplo, un lazo que conste de cien iteraciones se convertirá en cien bloques secuenciales de código, con cada uno de ellos ejecutando las mismas sentencias. La figura 1-2 ilustra gráficamente un desenrollo de lazos.

Obsérvese que la figura simplemente muestra el principio general del desenrollo de lazos. El método que emplee gcc puede ser drásticamente diferente.

Las optimizaciones que habilitan `-finline-functions` y `-funroll-loops` pueden mejorar en forma notable la velocidad de ejecución de un programa porque evitan la acumulación de llamadas a funciones y búsquedas de variables, pero el precio es habitualmente un gran incremento en el tamaño del archivo ejecutable o el archivo objeto. Es cuestión de experimentar para determinar si el aumento de velocidad de ejecución justifica el aumento del tamaño del archivo. Ver las páginas de información de gcc para obtener más detalles sobre los indicadores (*flags*) `-f` para el procesador.



**Figura 1.2.** El desenrolle de lazos expande los lazos de manera similar a la que el preprocesador expande los macros.

### CONSEJO

Generalmente, con un nivel de optimización `-O2` resulta suficiente. Aún en programas pequeños, tales como el de la primera sección “Ejemplo” al comienzo de este capítulo, el lector apreciará pequeñas reducciones en el tamaño del código y pequeños incrementos en el desempeño.



EJEMPLO

### Ejemplo

Esta situación utiliza el máximo nivel de optimización, `O3`, para compilar los listados 1.2, 1.3, y 1.4. Compare el tamaño del programa optimizado con el tamaño del programa no-optimizado.



SALIDA

```
$ gcc -O3 mensajes.c mostrar.c -o nuevo_hola
$ ls -l nuevo_hola
-rwxr-xr-x 1 kurt_wall usuarios 12091 Jul 13 21:06 nuevo_hola*
$ gcc -O0 mensajes.c mostrar.c -o nuevo_hola
$ ls -l nuevo_hola
-rwxr-xr-x 1 kurt_wall usuarios 12107 Jul 13 21:07 nuevo_hola*
```

La opción `-O0` desactiva toda optimización. Aunque la diferencia es pequeña, solo 16 bytes, debería resultar sencillo comprender que programas más grandes producirán mayores ahorros de espacio en disco.

### OPCIONES DE DEPURACIÓN

Los errores (*bugs*) son tan inevitables como la muerte o los impuestos. Para sobrellevar esta triste realidad, utilice las opciones `-g` y `-ggdb` de `gcc` para insertar información de depuración en sus programas compilados. Estas opciones facilitan las sesiones de depuración.

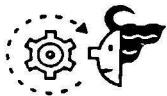
La opción `-g` admite tres niveles, 1, 2 o 3, que especifican cuánta información de depuración incluir. El nivel predeterminado es 2 (`-g2`), que incluye gran cantidad de tablas de símbolos, números de líneas e información sobre variables locales y externas. La información para depuración de nivel 3 incluye toda la información del nivel 2 y todas las definiciones de macros presentes. El nivel 1 genera sólo la información suficiente para crear rastreos hacia atrás y volcados de pila. No genera información de depuración para variables locales ni para números de línea.

Si el lector piensa emplear el depurador GNU, `gdb` (cubierto en el capítulo 20, “Un toolkit de depuración”), el empleo de la opción `-ggdb` crea información adicional que facilita las tareas de depuración bajo `gdb`. La opción `-ggdb` acepta el mismo nivel de especificaciones que `-g`, y éstas producen los mismos efectos sobre la salida del depurador. La utilización de cualquiera de las dos opciones de habilitación del depurador incrementará notablemente, sin embargo, el tamaño de su archivo ejecutable.



#### EJEMPLO

Este ejemplo le muestra cómo compilar con información de depuración e ilustra el impacto que los símbolos, que controlan la depuración, pueden tener en el tamaño de un archivo ejecutable. Una compilación y linkeo comunes del Listado 1.1 produjo un archivo binario de 4089 bytes en mi sistema. Los tamaños obtenidos cuando compilé el mismo código fuente con las opciones `-g` y `-ggdb` pueden llegar a sorprenderlo:



#### SALIDA

```
$ gcc -g hola.c -o hola_g
$ ls -l hola_g
-rwxr-xr-x 1 kurt_wall usuarios 6809 Jul 13 21:09 hola_g*
$ gcc -ggdb hola.c -o hola_ggdb
$ ls -l hola_ggdb
-rwxr-xr-x 1 kurt_wall usuarios 354867 Jul 13 21:09 hola_ggdb*
```

Como se podrá apreciar, la opción `-g` incrementó el tamaño del archivo binario resultante en un cincuenta por ciento, ¡mientras que la opción `-ggdb` infló el mismo archivo casi un novecientos por ciento! A pesar del aumento de tamaño, se debería proveer a los archivos ejecutables con símbolos estándar de depuración (creados utilizando `-g`) para el caso de que alguien encuentre algún problema y trate de depurar el código en lugar suyo.

---

### CONSEJO

Como regla general, depure primero, optimice después. No interprete, sin embargo, que "optimizar después" signifique "no tener en cuenta la eficiencia durante el proceso de diseño". La optimización, en este contexto, se refiere a las prestaciones que introduce el compilador, comentadas en esta sección. Un buen diseño y algoritmos eficientes tienen mucho mayor impacto en el desempeño global que lo que pueda lograr cualquier tipo de optimización lograda por el compilador. En verdad, si uno se toma el tiempo necesario para crear un diseño adecuado y utiliza algoritmos rápidos, puede no llegar a necesitar optimizar, aun cuando probar esto último nunca viene mal.

---

## Extensiones del GNU C

La implementación del GNU C extiende el ANSI C estándar de diversas maneras. Si al lector no le importa escribir un código que abiertamente no responda al estándar ANSI/ISO, algunas de estas extensiones pueden ser muy útiles. Para encontrar un tratamiento completo de las extensiones del proyecto GNU, el lector con inquietudes puede referirse a las páginas de información de gcc (en especial, pruebe el comando `info gcc "C Extensions"`). Las extensiones cubiertas en esta sección son las que se ven frecuentemente en los encabezados del kernel y el código fuente de Linux.

---

### PRECAUCIÓN

El problema con el código no estándar es doble. En primer lugar, si uno está utilizando extensiones GNU, su código sólo compilará adecuadamente, si es que directamente compila, con gcc. El otro problema es que los códigos no estándar no adhieren a las normas ANSI/ISO. Muchos usuarios utilizan en sus compiladores la menor cantidad posible de especificadores de compilación, de modo que los códigos fuente que no conformen a los estándares ANSI/ISO no compilarán. La política más inteligente, si uno desea incluir características no estándar en el código de su programa, es encerrarlas entre directivas `#ifdef` para permitir su compilación condicional. En el grupo de noticias `comp.lang.c` lo pondrían poco menos que en la lista negra por utilizar código no estándar.

---

Para proveer unidades de almacenamiento de 64 bits, por ejemplo, gcc ofrece el tipo `long long int`, utilizado en el Listado 1.5:

```
long long var_entera_larga;
```

---

### NOTA

El tipo `long long int` forma parte del borrador del nuevo estándar ISO para C. Este último es todavía un borrador, sin embargo, al cual muy pocos compiladores adhieren por ahora.

---

En la plataforma x86, esta definición da como resultado una ubicación de 64 bits en memoria denominada `long_int_var`. Otra extensión gcc que se encontrará en los archivos de encabezado de Linux es el empleo de funciones *inline*. Siempre que sea lo suficientemente corta, una función *inline* se expande en el código compilado casi de la misma forma en que lo hace un macro, eliminando así el costo de una llamada a una función. Las funciones *inline* son mejores que los macros, sin embargo, porque el compilador verifica su tipo durante la compilación. Para utilizar funciones *inline*, las tiene

que compilar con, por lo menos, una optimización `-O` y preceder la definición de la función con la palabra reservada `inline`, como se ilustra aquí:

```
inline int incrementar(int *a)
{
    (*a)++;
}
```

La palabra reservada `attribute` le suministra a gcc mayor información sobre su código y ayuda al optimizador de código a realizar su tarea. Para utilizar un atributo de función, la regla general es agregarle `__attribute__ ((attribute_name))` después del paréntesis de cierre de la declaración de la función en que se lo emplee.

Por ejemplo, las funciones estándar de biblioteca, tales como `exit` y `abort`, nunca retornan, así que el compilador puede generar un código ejecutable ligeramente más eficiente si se le informa que esas funciones no retornan. Por supuesto, los programas a nivel de usuario pueden definir también funciones que no retornen. gcc le permite a uno especificar el atributo `noreturn` para dichas funciones, que actúa como una señal para que el compilador optimice la función.

Así, supongamos que se tiene una función denominada `terminar_si_error`, que nunca retorna. Su declaración tendría el siguiente aspecto:

```
void terminar_si_error(void) __attribute__ ((noreturn));
```

Normalmente se la definiría así:

```
void terminar_si_error(void)
{
    /* el codigo va aqui */
    exit(1);
}
```

También se pueden aplicar atributos a las variables. El atributo `aligned` instruye al compilador para que alinee la ubicación en memoria de una variable sobre un límite de byte especificado. A la inversa, el atributo `packed` le indica a gcc que utilice la mínima cantidad de espacio requerida por variables convencionales o de tipo `struct`. Utilizado con variables de tipo `struct`, `packed` eliminará cualquier tipo de relleno que gcc normalmente insertaría con propósitos de alineación.

Una extensión terriblemente útil es la de rangos para las sentencias `case`. La sintaxis es similar a la siguiente:

```
case VALOR_INFERIOR ... VALOR_SUPERIOR:
```

Obsérvese que se requiere de al menos un espacio antes y después de los puntos suspensivos. Los rangos para `case` se utilizan en las sentencias `switch` para especificar valores que caen entre `valor_inferior` y `valor_superior`, como lo ilustra el siguiente fragmento de código:

```
switch(var_entera) {
    case 0 ... 2:
        /* el código va aquí */
        break;
    case 3 ... 5:
        ...
}
```

Este fragmento es equivalente a:

```
switch(var_entera) {
    case 1:
    case 2:
        /* el código va aquí */
        break;
    case 3:
    case 4:
    case 5:
        ...
}
```

Obviamente, el rango de `case` es simplemente una notación abreviada para la sintaxis tradicional de la sentencia `switch`, pero es realmente conveniente.



#### EJEMPLO

### Ejemplos

1. Este ejemplo ilustra el empleo de la palabra reservada `inline`. El programa itera por un lazo 10 veces, incrementando una variable con cada iteración.

```
/* Nombre del programa en Internet: inline.c */
/*
 * inline.c - Utilizacion de la palabra reservada inline, que permite
 * generar funciones expandibles durante la compilacion
 */
#include <stdio.h>
inline int incrementar(int *a)
{
```

```

        (*a)++;
    }

int main(void)
{
    int i = 0;
    while(i < 10) {
        incrementar(&i);
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}

```

En un ejemplo tan simple, es posible que no vea incrementos o disminuciones significativos en la velocidad o el tamaño del código, pero el ejemplo sirve para ilustrar cómo utilizar la palabra reservada `inline`.



**EJEMPLO**

2. Este ejemplo muestra cómo utilizar la palabra reservada `attribute` y el atributo `noreturn` y también cómo utilizar rangos para `case`.

```

/* Nombre del programa en Internet: noret.c */
/*

* no_retorno.c - Empleo del atributo noreturn
*/
#include <stdio.h>

void terminar_ahora(void) __attribute__((noreturn));

int main(void)
{
    int i = 0;

    while(1) {
        switch(i) {
            case 0 ... 5:
                printf("i = %d\n", i);
                break;

```

```

        default:
            terminar_ahora();
        }
        ++i;
    }
    return 0;
}
void terminar_ahora(void)
{
    puts("terminando ahora");
    exit(1);
}

```

Tenga cuidado, cuando redacte un código que tenga un lazo infinito como el lazo `while` de este programa, que exista un punto de salida, tal como salir del lazo cuando el contador del mismo alcance cierto valor. Para nuestros propósitos actuales, sin embargo, el ejemplo ilustra perfectamente cómo funciona la extensión que permite especificar el rango de `case`. La salida de este programa es:



```

$ ./no_retorno
i = 0
i = 1
i = 2
i = 3
i = 4
i = 5
terminando ahora

```

La extensión para la especificación del rango de `case` se comporta de la misma manera en que lo haría una sentencia `case` normal. Cuando `i` alcanza el valor 6, el flujo del programa llega al bloque `default`, se ejecuta `terminar_ahora`, y el programa termina.



3. El ejemplo final ilustra el efecto de utilizar el atributo `packed` con una estructura. El programa meramente imprime el tamaño de las estructuras y sus miembros constitutivos.

```

/* Nombre del programa en Internet: packme.c */
/*
 * compactar.c - Utilizacion del atributo packed de gcc
 */

```

```
#include <stdio.h>

/* una estructura "compactada" */
struct COMPACTADA {
    short primer_arreglo[3];
    long l;
    char segundo_arreglo[5];
} __attribute__((packed));

/* una estructura normal sin compactar */
struct NORMAL {
    short primer_arreglo[3];
    long l;
    char segundo_arreglo[5];
};

int main(void)
{
    struct COMPACTADA packed;
    struct NORMAL unpacked;

    /* imprimir el tamaño de cada miembro para propósitos de comparación */
    fprintf(stdout, "el tamaño de char segundo_arreglo[5] = %d bytes\n",
sizeof(char) * 5);
    fprintf(stdout, "          el tamaño de long = %d bytes\n", sizeof(long));
    fprintf(stdout, "    el tamaño de short primer_arreglo[3] = %d bytes\n",
sizeof(short) * 3);

    /* tamaños comparativos de las dos estructuras */
    fprintf(stdout, "          el tamaño de la estructura compactada = %d bytes\n",
sizeof packed);
    fprintf(stdout, "    el tamaño de la estructura sin compactar = %d bytes\n",
sizeof unpacked);

    return 0;
}
```

Para utilizar cualquiera de las extensiones de GNU al lenguaje C no se requiere ningún comando especial de compilación. Compilada y ejecutada, la salida de este programa es la siguiente:

```
S gcc compactar.c -o compactar
S ./compactar
el tamaño de char segundo_arreglo[5] = 5 bytes
    el tamaño de long = 4 bytes
el tamaño de short primer_arreglo[3] = 6 bytes
    el tamaño de la estructura compactada = 15 bytes
    el tamaño de la estructura sin compactar = 20 bytes
```

Los dos elementos fueron definidos con miembros idénticos. La única diferencia fue el agregado del atributo `packed` a la estructura `P`. Como lo deja en claro la estructura del programa, los miembros de cada una de las estructuras requieren 15 bytes de espacio de almacenamiento. En una arquitectura x86, la estructura normal, sin compactar, `UP`, aumenta en sólo 5 bytes, mientras que el tamaño de la estructura compactada, `P` es simplemente la suma de los tamaños de sus miembros. Aunque unos simples 5 bytes de memoria ahorrada en este caso puede no parecer demasiado, imagínese el lector que tuviera un arreglo de 1.000.000 de estructuras (caso posible en algunas aplicaciones): esto arrojaría casi 5 megabytes de ahorro de memoria RAM. La memoria ahorrada no sería poca en este caso, evidentemente.

---

#### NOTA

Los bytes de relleno en la estructura normal fueron probablemente el resultado de alinear `str` y `s` sobre límites adecuados, mediante el agregado de tres y dos bytes, respectivamente, a los tamaños verdaderos. ANSI permite a los que programan en lenguaje C rellenar todas las estructuras hasta hacerlas arrancar desde los orígenes naturales de los datos cuyas longitudes sean múltiplos de palabras en la arquitectura que los aloja. O sea que, aunque el atributo `packed` ofrece algunos beneficios obvios, los brinda a expensas de la portabilidad del programa.

---

## Utilización de egcs

`egcs` es el conjunto experimental (o mejorado) de programas del compilador GNU. Originalmente fue un emprendimiento experimental tendiente a acelerar el desarrollo del `gcc` y a expandir el entorno operativo del `gcc`. Otros objetivos del proyecto `egcs` (pronunciado asimismo `eggs`, sin la `c`) fue añadir nuevos lenguajes, nuevas optimizaciones y arquitecturas de procesador a `gcc`.

En abril de 1999, el proyecto GNU designó formalmente al comité ejecutivo del `egcs` como responsable oficial del mantenimiento del `gcc`. Este último ha sido desde entonces denominado Colección de Compiladores GNU. La cuestión clave de esta clase de historia es sugerir que, en algún momento, `egcs` puede muy bien fusionarse con `gcc`. Entretanto, sin embargo, `egcs` es un producto separado. Es también el compilador predeterminado para C/C++ en por lo menos dos distribuciones, de modo que resulta importante comprender sus diferencias y mejoras respecto de `gcc`. Para obtener más información sobre `egcs`, visite su sitio web en <http://egcs.cygnus.com/>.

---

**NOTA**

Parecería que la versión actual de egcs, la 1.1.2, sería la última versión de egcs como programa autónomo. El siguiente lanzamiento, de acuerdo con la información obtenible en el sitio web de egcs, será la versión 2.95 de gcc.

---

egcs es totalmente compatible hacia atrás con gcc, de modo que todo lo que se dijo en este capítulo sobre gcc también es de aplicación para egcs. En verdad, aunque esto no ha sido oficialmente establecido por el equipo de desarrollo del kernel, uno puede compilar el kernel del Linux 2.2.x con egcs sin experimentar mayores problemas y el archivo ejecutable que se obtendría debería funcionar bien. egcs representa una mejora significativa sobre el compilador estándar gcc, versión 2.7.2.3. El próximo punto cubre algunas de estas mejoras.

**Mejoras a gcc**

La mayoría de las mejoras de egcs respecto de gcc afecta la operación del propio compilador en lugar de la de su interfaz de línea de comandos, de modo que se hallan mucho más allá del alcance de este libro. Sin embargo se han efectuado también algunos cambios en la interfaz de línea de comandos, los cuales se cubren en este punto.

Se añadieron varias advertencias nuevas. `-Wundef` y `-Wno-def` generan advertencias cuando una directiva `#if` de preprocesador intenta evaluar un identificador no definido. `-Wsign-compare` hace que egcs (y gcc) generen una advertencia cuando el código compara un valor con signo con otro sin signo. El indicador de `-Wall` ha sido extendido de modo de hacer que egcs emita advertencias cuando los valores enteros son declarados implícitamente. De manera análoga, el indicador de advertencia `-Wimplicit-function-declarations` advierte en los casos de declaración implícita de funciones.

Se han efectuado también mejoras a la depuración y a la optimización. Por ejemplo, además de los argumentos numéricos del especificador de optimización, `-O`, se puede ahora utilizar `-Os` para asignar prioridad a las optimizaciones de tamaño de código sobre las optimizaciones de velocidad. La selección entre optimizaciones entre tamaño y velocidad depende en su mayor parte de si necesita un alto desempeño o conservar espacio en disco. En estos días, en que tanto el costo de la memoria RAM y de los discos rígidos es relativamente económico, resulta tarea sencilla seleccionar velocidad por sobre tamaño. El tamaño de un programa, sin embargo, debe ser considerado en término de sus requerimientos de almacenamiento en disco tanto como la memoria que requiere para ser ejecutado. En situaciones donde, tanto el tamaño de almacenamiento en disco como la memoria RAM utilizada se deben administrar prudentemente, seleccione reducir el tamaño en lugar de aumentar la velocidad porque el programa más rápido del mundo será inútil si no llegase a caber en el disco o no contase con memoria RAM suficiente en la computadora.

El especificador `-fstack-check` comprueba desbordes de pila en sistemas que carecen de una prestación de ese tipo. Para mejorar la capacidad de depuración se han agregado nuevos indicadores para reseñas de desempeño. Las reseñas de desempeño le permiten al programador determinar con pre-

cisión la rapidez con que se ejecuta su programa, a cuáles funciones llama más seguido, qué variables utiliza con mayor frecuencia e identificar secciones o variables del código que no sean empleadas (a menudo denominadas código muerto). Con esa información, uno puede modificar su código fuente a fin de lograr que su programa sea más eficiente. egcs también incluye apoyo integrado para la herramienta de verificación de memoria (Checker). A continuación se listan algunos indicadores (flags) adicionales que son específicos de cada procesador empleado:

- `-fprofile-arcs` — Permite a egcs contar el número de veces que se ejecuta una función.
- `-ftest-coverage` — Crea archivos de datos para la herramienta de análisis de cobertura de código gcov.
- `-fbranch-probabilities` — Mejora la predicción por parte de egcs de las ramificaciones de código.

Finalmente, la salida que producen las opciones `—help` ha sido sumamente mejorada.

### Lo que viene

En este capítulo, el lector ha aprendido cómo utilizar el compilador de C de GNU, gcc, y debido a su similitud con gcc, cómo utilizar egcs. La mayoría de los programas que uno construya, aun cuando sean programas de archivo único, van a necesitar de largos tiempos de compilación. Esta flexibilidad puede ser por otra parte cansadora y tediosa de tipear, de modo que el próximo capítulo le mostrará cómo automatizar el proceso de construcción de software utilizando otra gran herramienta de UNIX/Linux, make. Con un entendimiento básico de ambas herramientas, el lector estará en condiciones de continuar hacia los temas de programación de sistemas que le permitan comenzar a programar en Linux con ejemplos.